

Spring Data JPA

Complete Interview Preparation Guide

Deep Explanations · All Concepts · Real Examples · Why & When & How

Chapters	8 (Intro → Transactions)
Style	Concept first, code second
Level	Beginner → Senior Engineer
Goal	You explain every concept with confidence

CHAPTER

Table of Contents

Chapter 1 — Introduction to Spring Data JPA

- › The three-layer stack: JPA, Hibernate, Spring Data JPA
- › Why Spring Data JPA was invented — the boilerplate problem
- › Core advantages and real-world benefits
- › Architecture call-chain: service → repository → Hibernate → DB

Chapter 2 — Setting Up and Configuration

- › Maven / Gradle starter dependencies explained
- › Every application.properties key — what it does and why
- › What Spring Boot auto-configures for you and why
- › Multiple datasource pattern

Chapter 3 — Defining JPA Entities

- › What an entity is and how Hibernate tracks it
- › @Entity, @Table, @Id, @GeneratedValue — deep explanation
- › @Column constraints, @Embeddable value objects
- › @Enumerated pitfalls, lifecycle callbacks

Chapter 4 — Relationships Between Entities

- › One-to-One, One-to-Many, Many-to-Many — how FK ownership works
- › Why LAZY is always the right default — N+1 explained in depth
- › Cascade types — what each one does and when to use it
- › bidirectional relationships and helper methods

Chapter 5 — Repositories in Spring Data JPA

- › How Spring generates repository implementations at runtime
- › CrudRepository vs JpaRepository hierarchy explained
- › Derived query method naming — full grammar
- › @Query JPQL, native SQL, @Modifying
- › Projections and DTO-based results

Chapter 6 — Pagination and Sorting

- › Why pagination is critical in production
- › Pageable, PageRequest, Page, Slice — differences and internals
- › Sort class, multi-column sorting
- › REST controller wiring with @PageableDefault

Chapter 7 — Advanced Query Techniques

- › Specifications — the Composite pattern for dynamic queries
- › Criteria API internals
- › Advanced derived keywords

Chapter 8 — Transactions in Spring Data JPA

- › What a transaction is and why ACID matters
- › @Transactional internals — Spring proxy mechanism
- › All 7 propagation types with real-world use cases

- › Isolation levels explained
- › Read-only transactions, optimistic vs pessimistic locking

CHAPTER 1

Introduction to Spring Data JPA

1.1 The Problem Spring Data JPA Was Built to Solve

Before Spring Data JPA existed, every developer writing a data-access layer had to write the same repetitive boilerplate code over and over again. For every entity — say a User, an Order, a Product — you needed to manually write a DAO (Data Access Object) class that injected an EntityManager, opened transactions, called `em.persist()`, `em.find()`, `em.merge()`, `em.remove()`, and handled exceptions. If you had 30 entities, you wrote 30 almost-identical DAO classes. This was not only tedious but also a source of bugs, because developers would sometimes forget to close sessions, miss transaction boundaries, or write incorrect JPQL.

Spring Data JPA was created specifically to eliminate this problem. The core idea is simple but powerful: you define an interface that declares what data-access operations you need, and Spring generates a complete, working implementation of that interface at application startup — automatically, without any code from you. You never write a DAO class again. This is not code generation in the traditional sense (no files are generated on disk); instead, Spring uses Java's dynamic proxy mechanism to create an in-memory class at runtime that implements your interface.

■ **WHY THIS MATTERS:** Think of it like a contract system. You write the contract (the interface), and Spring fulfills it. You say 'I want a method that finds all users by email' and Spring figures out the SQL, the JPQL, the transaction — everything — and wires it all together.

1.2 The Three-Layer Stack — JPA, Hibernate, and Spring Data JPA

These three technologies are often confused with each other because they are deeply interrelated, but they exist at very different levels of abstraction. Understanding the distinction is one of the most common interview questions and is fundamental to debugging any production issue.

JPA — The Specification

JPA stands for Java Persistence API (now Jakarta Persistence API). It is not a library you can download and run — it is a specification, a set of rules and interfaces defined by the Java EE / Jakarta EE committee. JPA defines things like: what the `@Entity` annotation means, what the EntityManager interface looks like, how the persistence context works, and what JPQL (Java Persistence Query Language) syntax is. JPA itself has no runnable code. It is purely a contract.

The value of having a specification is portability. Because your code uses only JPA annotations like `@Entity` and `@Id`, you can theoretically switch from Hibernate to EclipseLink to OpenJPA without changing your entity classes. In practice, most teams use Hibernate and never switch, but the abstraction is still valuable because it prevents you from accidentally writing Hibernate-specific code everywhere.

Hibernate — The JPA Provider (The Real Engine)

Hibernate is the most popular implementation of the JPA specification. It is the actual engine that does the real work: it translates your JPQL queries and method calls into database-specific SQL, it manages the first-level cache (the persistence context), the second-level cache (shared cache across transactions), it handles dirty checking (detecting which entity fields changed and generating UPDATE statements automatically), it manages lazy loading via CGLIB or ByteBuddy proxies, and it handles database schema generation.

When you call `userRepository.save(user)`, Hibernate is the one actually executing `INSERT INTO users ...` or `UPDATE users SET ...` against your database. Spring Data JPA talks to JPA interfaces, and Hibernate implements those interfaces. Without Hibernate (or another JPA provider), Spring Data JPA cannot function.

Spring Data JPA — The Abstraction Layer

Spring Data JPA sits on top of both JPA and Hibernate. Its job is to remove the boilerplate of writing repository classes. It provides the Repository hierarchy (`CrudRepository`, `JpaRepository`), the mechanism that parses method names and converts them to queries, the `@Query` annotation support, pagination and sorting abstractions, Specifications for dynamic queries, and seamless integration with Spring's transaction management and dependency injection.

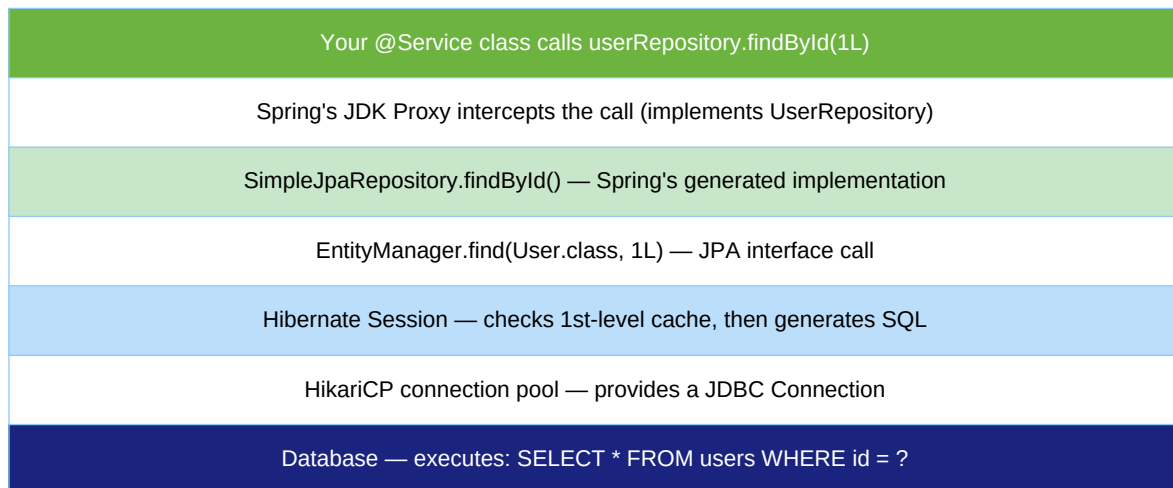
Layer	What It Is	Key Responsibility	Without It You Would...
JPA	Jakarta EE specification	Defines <code>@Entity</code> , EntityManager API, JPQL standard	Have to write portable code tied directly to Hibernate
Hibernate	JPA provider / ORM	Generates SQL, manages caches, dirty checking, lazy loading	Handle <code>ResultSet</code> , <code>PreparedStatement</code> , <code>ResultSet</code> , <code>PreparedStatement</code>
Spring Data JPA	Spring abstraction over JPA	Auto-generates repository impls, derived queries, pagination	Write all DAO classes with EntityManager injection

■ **INTERVIEW TIP:** In an interview, if asked 'what is the difference between JPA, Hibernate, and Spring Data JPA', answer: JPA is the specification (the contract), Hibernate is the implementation of that contract (the engine that actually talks to the database), and Spring Data JPA is a higher-level framework that sits on top of both and removes the need to write repository boilerplate code. They are layered, not competing.

1.3 How Spring Data JPA Works Internally — The Call Chain

When you write an interface like `public interface UserRepository extends JpaRepository<User, Long>`, Spring scans for it at startup. It finds that it extends a Spring Data repository interface, inspects all the method signatures, and uses a `RepositoryFactoryBean` to create a JDK dynamic proxy that implements your interface. The proxy delegates all method calls to a `SimpleJpaRepository` instance, which is Spring Data JPA's built-in implementation class containing the actual `EntityManager` calls.

When you call `userRepository.findById(1L)`, the proxy intercepts that call, finds the corresponding method in `SimpleJpaRepository`, which calls `entityManager.find(User.class, 1L)`, which Hibernate intercepts, checks its first-level cache (the current persistence context), and if not found there, executes `SELECT * FROM users WHERE id = 1` against the database. The result comes back up through every layer in reverse order.



This layered architecture is the reason why debugging Spring Data JPA issues requires understanding all three layers. A `LazyInitializationException` is a Hibernate concept. A duplicate key violation comes from the database. A missing `@Transactional` is a Spring issue. Knowing which layer to look at saves enormous time during debugging.

CHAPTER 2

Setting Up and Configuration

2.1 The Spring Boot Starter — What It Actually Pulls In

When you add `spring-boot-starter-data-jpa` to your project, you are not just adding one library. This single starter dependency is a carefully curated bundle that transitively includes: `spring-data-jpa` (the main Spring Data JPA library), `spring-orm` (Spring's ORM integration layer), `hibernate-core` (the JPA provider), `jakarta.persistence-api` (the JPA spec interfaces), `spring-tx` (transaction management), `HikariCP` (the default connection pool), and `spring-jdbc`. Understanding this matters because when you see an error from `'org.hibernate.*'` or `'com.zaxxer.hikari.*'`, you know those classes are coming from this starter.

nom.xml (Maven)

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-ina</artifactId>
<!-- No version needed - Spring Boot BOM manages it -->
</dependency>
<!-- You still need your database driver separately -->
<dependency>
<groupId>org.postgresql</groupId>
<artifactId>postgresql</artifactId>
<scope>runtime</scope> <!-- only needed at runtime not compile time -->
</dependency>
```

build.gradle (Gradle Kotlin DSL)

```
implementation("org.springframework.boot:spring-boot-starter-data-jpa")
runtimeOnly("org.postgresql:postgresql")
```

■ **WHY THIS MATTERS:** The starter uses Spring Boot's BOM (Bill of Materials) to manage all version numbers. You should never manually specify versions for Spring libraries — let the BOM ensure compatibility. Mismatched Hibernate and Spring Data JPA versions are a common source of mysterious startup errors.

2.2 application.properties — Every Property Explained in Depth

The `application.properties` file is where you configure how Spring connects to your database and how Hibernate behaves. Every property here has a specific reason for existing, and understanding each one helps you tune your application for production.

application properties — fully annotated

[illegible]

[illegible]

■ **WATCH OUT:** Never use `ddl-auto=create` or `create-drop` in production. These will drop your tables on every startup, destroying all data. In production, always use `validate` or `none`, and manage schema changes with Flyway or Liquibase.

2.3 What Spring Boot Auto-Configures and Why You Should Care

One of Spring Boot's most powerful features is auto-configuration. When `spring-boot-starter-data-jpa` is on the classpath, Spring Boot's `JpaRepositoriesAutoConfiguration` and `HibernateJpaAutoConfiguration` classes automatically create all the beans you need for a working JPA setup. Understanding what gets created helps you know what to override when you have special requirements.

- **DataSource bean:** HikariCP connection pool, configured from `spring.datasource.*` properties. HikariCP is chosen because it is the fastest and most stable connection pool available for the JVM.
- **EntityManagerFactory bean:** The root JPA concept. Spring Boot creates a `LocalContainerEntityManagerFactoryBean` that scans your `@SpringBootApplication` package (and sub-packages) for `@Entity` classes and registers them all. This scan is why your entities must be in the same package hierarchy as your main class.

- **JpaTransactionManager bean:** Implements Spring's PlatformTransactionManager interface. This is what @Transactional talks to. It knows how to begin, commit, and roll back JPA (Hibernate) transactions.

- **Repository proxy beans:** Spring Data's RepositoryComponentProvider scans for interfaces extending Repository (or its sub-interfaces) and creates proxy implementations for each one, registering them as Spring beans. This is why you can @Autowire UserRepository even though you never wrote an implementing class.

■ **WHY THIS MATTERS:** When you have only one datasource, all of this is completely automatic — zero configuration needed beyond application.properties. You only need to create custom beans when you have multiple datasources, non-standard entity locations, custom naming strategies, or when you need to tune Hibernate behaviour.

CHAPTER 3

Defining JPA Entities

3.1 What Is a JPA Entity and How Does Hibernate Track It?

A JPA Entity is a Java class that Hibernate maps to a database table. Each instance of the class represents one row in that table. When you mark a class with `@Entity`, you are telling Hibernate three things: first, this class should be mapped to a database table; second, instances of this class should be managed by the `EntityManager` (the persistence context); third, Hibernate should generate the appropriate SQL for INSERT, UPDATE, DELETE, and SELECT operations on this class.

Hibernate tracks entities through a concept called the Persistence Context. Every `EntityManager` has exactly one persistence context — think of it as a first-level cache and change-tracker. When you load an entity within a transaction, Hibernate takes a snapshot of its fields. When the transaction commits, Hibernate compares the current field values to the original snapshot (this is called dirty checking) and automatically generates UPDATE statements for any fields that changed. This is why you do NOT need to call `repository.save()` after modifying an entity that is already loaded within a transaction — Hibernate will detect the change and save it automatically.

■ **WHY THIS MATTERS:** This dirty-checking behaviour surprises many beginners. If you load a `User` entity within a `@Transactional` service method and change `user.setName('NewName')`, Hibernate will generate an UPDATE statement at the end of the transaction even if you never called `userRepository.save(user)`. This is by design and is a core JPA feature.

3.2 @Entity and @Table — Why Both Exist

`@Entity` is the fundamental JPA annotation that marks the class as a persistent entity. Without it, Hibernate completely ignores the class. `@Table` is optional — if you omit it, Hibernate uses the class name as the table name. You add `@Table` when you need to control the exact table name (especially useful when the Java naming convention differs from your DBA's SQL naming convention), when you need to add database-level constraints like indexes, or when you need to specify a schema.

Full Entity with every annotation explained

```
@Entity // Required: tells Hibernate to manage this class
@Table(
    name = "users" // Explicit table name (overrides default 'user' class name)
    schema = "app" // Optional: put table in a specific DB schema
    indexes = {
        // Creates an actual index in the DB when using ddl-auto=create/update
        @Index(name = "idx user email" columnList = "email" unique = true)
        @Index(name = "idx user role" columnList = "role")
    }
    uniqueConstraints = {
        @UniqueConstraint(columnNames = {"first name" "last name"})
    }
)
public class User {
    @Id // Required: marks the primary key field
```

```

@GeneratedValue(strategy = GenerationType.IDENTITY)
// IDENTITY = DB auto-increment (SERIAL in PostgreSQL, AUTO_INCREMENT in MySQL)
private Long id;

@Column(
    name = "full name" // Map to different column name than field name
    nullable = false // Creates NOT NULL constraint in DB
    length = 100 // Creates VARCHAR(100) instead of VARCHAR(255)
    updatable = false // This column value can never be changed after insert
)
private String name;

@Column(unique = true) // Adds a UNIQUE constraint on just this column
private String email;

@Enumerated(EnumType.STRING) // Store as 'ADMIN' 'USER' — NOT 0 1 2
@Column(nullable = false)
private Role role;

@Embedded // Address fields stored in SAME table (not a join)
private Address address;

@CreatedDate // Auto-populated by Spring Data JPA Auditing
@Column(updatable = false)
private LocalDateTime createdAt;

@LastModifiedDate // Auto-updated on every save
private LocalDateTime updatedAt;

@Version // Optimistic locking — Hibernate manages this
private Integer version;

// IMPORTANT: JPA requires a no-arg constructor (can be protected)
protected User() {}

public User(String name, String email) {
    // ...
}

```

3.3 @GeneratedValue Strategies — Choosing the Right One

The @GeneratedValue annotation tells Hibernate how to generate primary key values. This choice has significant performance implications, especially for applications that insert many records. There are four standard strategies, each with different behaviour.

GenerationType.IDENTITY — The Simple Default

With IDENTITY, Hibernate delegates primary key generation entirely to the database column's auto-increment feature (SERIAL in PostgreSQL, AUTO_INCREMENT in MySQL). After each INSERT, Hibernate executes a SELECT LAST_INSERT_ID() (or equivalent) to get the generated ID. This is the simplest strategy and works well for most applications. However, there is an important limitation: because Hibernate does not know the ID until after the INSERT is committed, it cannot batch multiple INSERT statements together. Each record must be inserted one at a time. For high-throughput insert scenarios (thousands of records per second), this is a significant performance bottleneck.

GenerationType.SEQUENCE — The Production Best Practice

With SEQUENCE, Hibernate uses a database sequence object to generate IDs. The key performance advantage is that Hibernate can pre-allocate a range of IDs in a single database round-trip. By setting allocationSize=50, Hibernate fetches the next sequence value and then locally assigns the next 50 IDs without touching the database again. This allows Hibernate to collect 50 INSERT statements and send them to the database as a batch, drastically reducing round-trips. This

is the recommended strategy for PostgreSQL and Oracle applications with high write throughput.

SEQUENCE strategy with custom allocation size

```
@Id
@GeneratedValue(
    strategy = GenerationType.SEQUENCE,
    generator = "user_seq"
)
@SequenceGenerator(
    name = "user_seq",
    sequenceName = "users_id_seq" // Name of the DB sequence object
    allocationSize = 50 // Pre-allocate 50 IDs per DB call
)
private Long id;
```

Strategy	ID generation	Batch insert?	Best for
IDENTITY	DB auto-increment column	No — must flush each insert	Simple apps, MySQL
SEQUENCE	DB sequence + local pool	Yes — batches possible	High-throughput PostgreSQL/Oracle
TABLE	Separate 'id_table' row	No — row locking	Legacy portability (avoid in new code)
UUID	App-generated UUID	Yes	Distributed systems, no DB round-trip

3.4 @Embeddable — Modelling Value Objects

In Domain-Driven Design, a Value Object is a concept that has no identity of its own — it only has meaning in the context of an entity. An address, for example, has no 'id' — it belongs to a person. In JPA, @Embeddable lets you model value objects as separate Java classes while storing their data in the owning entity's table (no join, no foreign key). This gives you better Java type safety and object modelling without any performance cost.

@Embeddable value object example

```
@Embeddable // No @Entity, no @Id — not a table, just a reusable set of columns
public class Address {
    @Column(name = "street") private String street;
    @Column(name = "city") private String city;
    @Column(name = "zip code") private String zipCode;
    @Column(name = "country") private String country;
}

// Using it in an entity — all Address fields go INTO the users table
@Entity @Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Embedded
    private Address homeAddress;
    // Use @AttributeOverrides if you embed Address twice
    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street", column = @Column(name = "work_street")),
        @AttributeOverride(name = "city", column = @Column(name = "work_city")),
        @AttributeOverride(name = "zipCode", column = @Column(name = "work_zip")),
        @AttributeOverride(name = "country", column = @Column(name = "work_country"))
    })
    private Address workAddress;
```

```
}  
// Resulting table has: id street city zip code country  
// work_street, work_city, work_zip, work_country
```

3.5 @Enumerated — Why STRING Is Non-Negotiable

@Enumerated has two options: EnumType.ORDINAL and EnumType.STRING. ORDINAL stores the enum's position in its declaration as an integer (0, 1, 2...). STRING stores the enum's name as a VARCHAR. This sounds like a minor implementation detail, but the choice of ORDINAL has caused real production disasters.

Imagine you have enum Role { ADMIN, USER, MODERATOR } and you are using ORDINAL. ADMIN is stored as 0, USER as 1, MODERATOR as 2. Six months later, a developer adds a new role: enum Role { ADMIN, GUEST, USER, MODERATOR }. Now USER is 2 and MODERATOR is 3. Every existing 'USER' record in the database (stored as 1) now maps to GUEST. Every MODERATOR (stored as 2) now maps to USER. The data is silently, completely corrupted — and there is no error message, no exception. Just wrong data serving the wrong users the wrong permissions.

■ **WATCH OUT:** ALWAYS use @Enumerated(EnumType.STRING). Never use ORDINAL in any production system. The slightly larger storage cost (VARCHAR vs INT) is completely irrelevant compared to the data safety you gain.

CHAPTER 4

Relationships Between Entities

4.1 Understanding Relationship Ownership — The Most Confusing Concept

Every relationship in JPA has an 'owning side' and an 'inverse side'. This is one of the most confusing concepts for beginners but is absolutely critical to understand, because the owning side determines which entity's table holds the foreign key column, and more importantly, only changes made to the owning side are actually persisted to the database.

The owning side is always the entity that has the `@JoinColumn` annotation (for one-to-one or many-to-one), or the `@JoinTable` annotation (for many-to-many). The inverse side uses the 'mappedBy' attribute to say: 'the FK is managed by the other side — look there.' If you only update the inverse side of a bidirectional relationship and forget to update the owning side, Hibernate will not persist the relationship change to the database.

4.2 One-to-Many and Many-to-One — The Most Common Relationship

One-to-Many is the most common relationship type you will encounter. A customer can have many orders. A post can have many comments. A department can have many employees. In SQL, this is represented by a foreign key column in the 'many' table pointing to the 'one' table. In JPA, the entity with the foreign key column is always the 'many' side (also called the owning side), and it uses `@ManyToOne` with `@JoinColumn`. The 'one' side uses `@OneToMany` with `mappedBy`.

Order (one) → OrderItem (many) — bidirectional

```
@Entity @Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String customerName;
    // mappedBy = 'order' means: do look at OrderItem order field for the FK
    // This side does NOT own the relationship — it is the inverse side
    // cascade = ALL: when we save/delete an Order cascade to its items
    // orphanRemoval = true: if we remove an item from the list DELETE it from DB
    @OneToMany(
        mappedBy = "order"
        cascade = CascadeType.ALL
        orphanRemoval = true
    )
    private List<OrderItem> items = new ArrayList<>();
    // ALWAYS use helper methods for bidirectional relationships!
    // If you just do items.add(item) without item.setOrder(this)
    // the FK column in order items will be NULL — relationship not persisted
    public void addItem(OrderItem item) {
        items.add(item); // update inverse side
        item.setOrder(this); // update OWNING side — this is what actually persists
    }
    public void removeItem(OrderItem item) {
```

```

items.remove(item);
item.setOrder(null);
}
}

@Entity @Table(name = "order_items")
public class OrderItem {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // This side OWNS the relationship - the FK column 'order_id' lives HERE
    // FetchType LAZY: do not load the Order object unless it is actually accessed
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;

    private int quantity;

    private BigDecimal price;
}

```

4.3 Lazy vs Eager Fetching — The Most Critical Performance Topic

Fetch type controls WHEN Hibernate loads associated entities. This is not a minor configuration detail — it is one of the most important performance decisions in your application, and misunderstanding it is the root cause of the infamous N+1 query problem, which is the single most common performance bug in Spring applications.

What LAZY Fetching Means

When a relationship is marked `FetchType.LAZY`, Hibernate does NOT load the associated entities when you load the parent. Instead, it creates a lightweight proxy object (a fake stand-in). The actual database query to load the association only fires when you first access a method on that proxy — for example, when you call `order.getItems()` or `order.getItems().size()`. This is on-demand loading, and it is the right default because you often load entities without needing their associations.

What EAGER Fetching Means

When a relationship is marked `FetchType.EAGER`, Hibernate immediately executes a JOIN (or an additional SELECT) to load the association every time you load the parent — regardless of whether you will ever use the association. If you call `orderRepository.findAll()` to display a list of order summaries on a screen, and `Order` has an `EAGER` association to a 100-item list of `OrderItems`, Hibernate loads ALL of those items for ALL of those orders, even though you are only displaying the order number and date. This wastes database resources, memory, and time.

The N+1 Problem — Explained Clearly

The N+1 problem occurs when Hibernate executes 1 query to load a list of N parent entities, and then executes 1 additional query for EACH parent to load a LAZY association — resulting in N+1 total queries instead of 1 or 2. It is called N+1 because if you have 100 orders, you get 101 queries: 1 for the orders list, then 100 for each order's items.

N+1 problem and solutions

[illegible]

```

System.out.println(o.getItems().size()); // Query 2: N+1: SELECT items
}
// With 100 orders: 101 queries fired. With 1000 orders: 1001 queries!
// ■■■■ SOLUTION 1: JOIN FETCH in @Query ■■■■
@Query("SELECT DISTINCT o FROM Order o JOIN FETCH o items")
List<Order> findAllWithItems();
// Result: single query with JOIN — just 1 query for all orders + all items
// ■■■■ SOLUTION 2: @EntityGraph ■■■■
// Avoids writing JPQL — declarative way to eager-load for specific method
@EntityGraph(attributePaths = {"items", "items.product"})
List<Order> findByCustomerName(String name);
// ■■■■ SOLUTION 3: DTO Projection (best for read-only views) ■■■■
@Query("SELECT new com.app.dto.OrderSummary(o.id, o.customerName, SIZE(o.items))"
+ " FROM Order o GROUP BY o.id, o.customerName")
List<OrderSummary> findOrderSummaries();

```

■ **INTERVIEW TIP:** In interviews, when asked about performance issues, always mention N+1. Explain it clearly: '1 query loads N entities, then N additional queries load each entity's lazy association — N+1 total.' Then explain that JOIN FETCH or @EntityGraph is the fix. This shows you understand both Hibernate internals and real-world performance engineering.

4.4 Cascade Types — What They Mean and When To Use Each

Cascade types tell Hibernate: when you perform an operation (save, delete, refresh) on the parent entity, should that operation be propagated (cascaded) to the child entities? Without cascade, you would have to manually save and delete every child entity individually. With cascade, operations on the parent automatically apply to children. Understanding each cascade type is essential because using the wrong one can cause accidental data deletion.

- **CascadeType.PERSIST:** When you call `entityManager.persist(parent)` (or `repository.save()` on a new entity), Hibernate will also persist all new child entities in the collection. Without this, saving a new Order with new OrderItems would require you to manually save each OrderItem first.
- **CascadeType.MERGE:** When you call `entityManager.merge(parent)` (or `save()` on a detached entity), Hibernate will also merge the child entities. Used when you pass a detached object back into the persistence context after the original transaction ended.
- **CascadeType.REMOVE:** When you delete the parent entity, Hibernate also deletes all child entities. Use this carefully — on a @OneToMany from Order to OrderItems, this makes sense (deleting an order should delete its items). But cascading REMOVE through a @ManyToMany is dangerous.
- **CascadeType.REFRESH:** When you call `refresh()` to reload the parent's state from the database (discarding changes), Hibernate also refreshes child entities. Useful for clearing changes made within a transaction.
- **CascadeType.DETACH:** When the parent is detached from the persistence context, children are also detached. Rarely needed explicitly.
- **CascadeType.ALL:** Shorthand for all five types above. Use on @OneToMany when the child entity has no meaning outside its parent (OrderItem only exists for an Order). Do not use on @ManyToMany.

■ **WATCH OUT:** Never use `CascadeType.ALL` or `CascadeType.REMOVE` on a @ManyToMany relationship. If Student has many Courses and you cascade REMOVE, deleting one Student will cascade

to Course entities — which deletes every other Student's enrollment in those courses. This is a common data-destruction bug.

CHAPTER 5

Repositories in Spring Data JPA

5.1 The Repository Hierarchy — Why Each Interface Exists

Spring Data JPA provides a hierarchy of repository interfaces, each adding more capabilities than the one below it. Understanding why this hierarchy exists — not just what methods each interface provides — is important both for designing clean code and for interview discussions.

The hierarchy follows the Interface Segregation Principle from SOLID design. Not every service needs full JPA capabilities. If a component only needs to save and retrieve objects, it should not be forced to depend on pagination and sorting methods it never uses. The hierarchy lets you choose the minimal interface that matches your needs.

```
Repository<T, ID> ← Marker interface. No methods. Just tells Spring 'scan this interface.'

CrudRepository<T, ID> ← +save, saveAll, findById, findAll, existsById, count, delete, deleteAll

PagingAndSortingRepository<T, ID> ← +findAll(Sort), findAll(Pageable)

JpaRepository<T, ID> ← +flush, saveAndFlush, saveAllAndFlush, deleteAllInBatch
```

■ **WHY THIS MATTERS:** In a Spring Boot project, you should almost always extend `JpaRepository`. It gives you everything. The only reason to use `CrudRepository` is if you want to shield a domain module from JPA-specific methods like `flush()` and `deleteAllInBatch()`, enforcing a cleaner architectural boundary.

5.2 How Derived Query Methods Work Internally

When Spring starts up and creates the proxy implementation for your repository interface, it inspects every method that is not one of the standard CRUD methods. For each custom method, it uses a class called `PartTreeJpaQuery` to parse the method name according to a strict naming grammar. It tokenizes the name, identifies the subject (`find`, `count`, `exists`, `delete`), the criteria (`ByEmail`, `AndAge`, `GreaterThan`), and any modifiers (`OrderBy`, `Asc`, `Desc`, `IgnoreCase`). It then constructs a JPQL query tree and compiles it into a Hibernate query. All of this parsing and compilation happens once at startup — not at runtime when the method is called. If your method name is grammatically incorrect (e.g., referencing a field that does not exist on the entity), Spring will throw an exception on startup, which is actually a great safety feature.

UserRepository.java — derived query examples with explanation

```
public interface UserRepository extends JpaRepository<User, Long> {
    // 'findBy' + 'Email' → SELECT u FROM User u WHERE u.email = :email
    Optional<User> findByEmail(String email);
    // 'findBy' + 'Name' + 'And' + 'Active' → WHERE name=? AND active=?
    List<User> findByNameAndActive(String name, boolean active);
    // 'GreaterThan' keyword → WHERE u.age > :age
```

```

list<User> findRvAgeGreaterThan(int age) {
    // 'Between' → WHERE u salary BETWEEN 'min AND 'max
    list<User> findRvSalaryBetween(BigDecimal min BigDecimal max) {
        // 'Containing' + 'IgnoreCase' → WHERE LOWER(u name) LIKE '%keyword%'
        list<User> findRvNameContainingIgnoreCase(String keyword) {
            // 'In' → WHERE u status IN ('statuses)
            list<User> findRvStatusIn(list<Status> statuses) {
                // 'IsNull' → WHERE u deletedAt IS NULL
                list<User> findRvDeletedAtIsNull() {
                    // 'True' → WHERE u active = true
                    list<User> findRvActiveTrue() {
                        // 'count' subject → SELECT COUNT(u) FROM User u WHERE
                        long countRvRole(Role role) {
                            // 'exists' subject → SELECT CASE WHEN COUNT > 0 THEN TRUE
                            boolean existsRvEmail(String email) {
                                // 'delete' subject → DELETE FROM users WHERE active = false
                                @Transactional
                                void deleteRvActiveFalse() {
                                    // 'findFirst3' → adds LIMIT 3 to the query
                                    list<User> findFirst3RvOrderRvCreatedAtDesc() {
                                        // Combined sort in method name
                                        list<User> findRvRoleOrderRvNameAscCreatedAtDesc(Role role) {
                                            }
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```

5.3 @Query Annotation — When Derived Methods Are Not Enough

Derived query methods are excellent for simple, single-table queries with straightforward conditions. However, they have real limitations: they cannot express aggregations (GROUP BY, HAVING), complex JOINS, subqueries, or window functions. For these cases, @Query lets you write explicit JPQL or native SQL. The important distinction between JPQL and native SQL is that JPQL works with entity class names and field names (Java names), while native SQL works with actual table and column names in the database.

```
@Query — JPQL native SQL and @Modifying
```

```
// ■■■ JPQL : uses entity/field names not table/column names ■■■  
// User is the entity class name; role and active are Java field names  
@Query("SELECT u FROM User u WHERE u.role = :role AND u.active = true")  
List<User> findActiveByRole(@Param("role") Role role);  
  
// ■■■ JPQL with JOIN ■■■  
@Query("SELECT DISTINCT o FROM Order o JOIN FETCH o.items i WHERE o.status = :s")  
List<Order> findOrdersWithItemsByStatus(@Param("s") OrderStatus s);  
  
// ■■■ Native SQL : uses real table/column names ■■■  
// Useful for DB-specific functions or when JPQL can't express the query  
@Query(  
    value = "SELECT u.* FROM users u WHERE u.created_at > :since" +  
        " AND u.role = :role ORDER BY u.name",  
    nativeQuery = true  
)  
  
List<User> findRecentByRole(@Param("since") LocalDateTime since,  
    @Param("role") String role);  
  
// ■■■ @Modifying: required for UPDATE and DELETE statements ■■■  
// Without @Modifying, Spring Data will try to parse as a SELECT and fail.  
// clearAutomatically = true: clear persistence context after update  
// (prevents stale cached entities from being returned in the same TX)  
@Modifying(clearAutomatically = true)  
@Transactional  
  
@Query("UPDATE User u SET u.active = false WHERE u.lastLogin < :cutoff")
```

```
int deactivateInactiveUsers(@Param("cutoff") LocalDateTime cutoff) {
    // Returns int = number of rows affected
}
```

5.4 Projections and DTO Results — Returning Only What You Need

When you call `userRepository.findAll()` and your `User` entity has 30 fields, Hibernate loads all 30 fields even if you only need name and email for a dropdown list. This wastes database I/O, JDBC result set processing, memory, and serialization time. Projections solve this by letting you define exactly which fields to return, resulting in a `SELECT` with only the columns you need.

There are three projection mechanisms in Spring Data JPA, each with different trade-offs. Interface projections are the most concise and easiest to use. Class-based (DTO) projections using constructor expressions in JPQL give you more control and avoid the proxy overhead. Dynamic projections let the caller decide the return type at the call site.

Three projection approaches — side by side

[illegible]

■ **INTERVIEW TIP:** For read-heavy APIs (like REST endpoints that serve lists), always use DTO or interface projections. They prevent: (a) loading unnecessary columns, (b) accidental lazy-loading outside transactions, and (c) serialization of sensitive fields. This is a senior-level best practice that interviewers love to hear.

CHAPTER 6

Pagination and Sorting

6.1 Why Pagination Is Not Optional in Production

Every developer who has worked on a production system has seen what happens when a table grows to millions of rows and someone calls `findAll()` without pagination. The database executes a full table scan, transfers millions of rows over the network, the JVM allocates gigabytes of memory to hold them all as entity objects, the garbage collector starts struggling, the HTTP response takes 30 seconds or more, and sometimes the application runs out of memory and crashes. Pagination is not an optimization — it is a fundamental requirement for any table that will grow beyond a few thousand rows.

Spring Data JPA's pagination support is built around two key abstractions: the `Pageable` interface (which describes WHICH page you want — page number, page size, and sort order) and the `Page` interface (which contains the results along with metadata about the total number of pages and items). Understanding how these fit together is essential for building scalable REST APIs.

6.2 Pageable — How to Construct and Use It

`Pageable` is an interface with three pieces of information: the page number (zero-based — page 0 is the first page), the page size (how many items per page), and an optional `Sort` specification. You create `Pageable` instances using the `PageRequest.of()` factory method. `PageRequest` is the main concrete implementation of `Pageable`.

Creating Pageable objects

```
// Basic: page 0, 10 items per page, no sort
Pageable p1 = PageRequest.of(0, 10);

// With single-column sort: page 0, 20 items, sorted by 'name' ascending
Pageable p2 = PageRequest.of(0, 20, Sort.by("name").ascending());

// With multi-column sort: sort by lastName ASC then createdAt DESC
Sort sort = Sort.by(
    Sort.Order.asc("lastName"),
    Sort.Order.desc("createdAt")
);
Sort.Order.asc("id") // tiebreaker — always add a stable sort column!
);
Pageable p3 = PageRequest.of(pageNumber, pageSize, sort);
// Sort.by() with direction shorthand
Pageable p4 = PageRequest.of(0, 10,
    Sort.by(Sort.Direction.DESC, "createdAt", "name"));
```

■ **NOTE:** Always add a stable, unique sort column (like 'id') as the last sort criterion. Without it, if two rows have identical values in your sort columns, their relative order is non-deterministic — you can get duplicate items across pages.

6.3 Page vs Slice — A Critical Design Decision

Page<T> executes TWO queries against the database: one query to fetch the current page of results (with LIMIT and OFFSET), and a second COUNT(*) query to find the total number of matching rows. The total count is necessary to calculate totalPages, so UI components like 'Page 3 of 47' can display correctly. The COUNT(*) query can be expensive on large tables, especially with complex WHERE clauses or JOINS.

Feature	Page<T>T<T>;	Slice<T>T<T>;
DB queries per call	2 (SELECT + COUNT)	1 (SELECT pageSize+1 rows)
Total count available	Yes — getTotalElements(), getTotalPages()	No
hasNext() available	Yes	Yes
Best for	Traditional paginated tables with page numbers	Infinite scroll, 'Load More', streaming
Performance	Slower on large tables (COUNT can be expensive)	Faster (no COUNT query)

```
// ■■■ Repository ■■■
public interface ProductRepository extends JpaRepository<Product, Long> {
    // Returns Page – includes total count
    Page<Product> findByCategory(String category, Pageable pageable);
    // Returns Slice – no total count, faster
    Slice<Product> findByPriceLessThan(BigDecimal maxPrice, Pageable pageable);
}

// ■■■ Service ■■■
@Service @Transactional(readOnly = true)
public class ProductService {
    public Page<ProductDto> getProductsByCategory(
        String category, int page, int size, String sortBy) {
        Pageable pageable = PageRequest.of(page, size,
            Sort.by(sorts).ascending());
        return productRepo.findByCategory(category, pageable)
            .map(ProductDto::fromEntity); // transform each item
    }
}

// ■■■ REST Controller ■■■
// Spring MVC auto-resolves Pageable from query params when
// @EnableSpringDataWebSupport is active (auto in Spring Boot)
// URI: GET /products?page=0&size=20&sort=name asc&sort=price desc
@GetMapping("/products")
public Page<ProductDto> listProducts(
    @PageableDefault(size = 20, sort = "name") Pageable pageable,
    @RequestParam String category) {
    return productService.getProductsByCategory(category, pageable);
}
```

CHAPTER 7

Advanced Query Techniques

7.1 Specifications — Dynamic Queries Without String Concatenation

Imagine you are building a user search feature. The search form has 8 optional fields: name, email, role, active status, minimum age, maximum age, creation date range, and department. Some users fill in 1 field, others fill in 7. You cannot write a derived query method for every combination (that would require $2^8 = 256$ methods). You could write a native SQL query that builds a string with IF conditions, but that is fragile, hard to test, and vulnerable to SQL injection. Specifications provide the clean, type-safe solution.

The Specification pattern (part of the Criteria API) represents a single predicate as an object. You create one Specification per filter condition, and then combine them with `.and()`, `.or()`, and `.not()` operators. This is an application of the Composite design pattern — simple predicates compose into complex queries while keeping each individual filter independently testable and reusable.

A `Specification<T>` is a functional interface with a single method: `Predicate toPredicate(Root<T> root, CriteriaQuery<?> query, CriteriaBuilder cb)`. The `Root` gives you access to the entity's fields. The `CriteriaBuilder` gives you factory methods to create predicates (`cb.equal`, `cb.like`, `cb.greaterThan`, etc.). Returning null from `toPredicate` means 'no condition' — Spring Data will ignore null predicates when building the final WHERE clause.

UserSpecifications.java — full dynamic search

```
public class UserSpecs {
    // Each method returns a Specification — a reusable, composable predicate
    public static Specification<User> hasName(String name) {
        return (root, query, cb) -> {
            if (name == null || name.isBlank()) return null; // no filter
            // LIKE %name% case-insensitive
            return cb.like(cb.lower(root.get("name")),
                "%" + name.toLowerCase() + "%");
        };
    }

    public static Specification<User> hasRole(Role role) {
        return (root, query, cb) -> {
            role == null ? null : cb.equal(root.get("role"), role);
        };
    }

    public static Specification<User> isActive(Boolean active) {
        return (root, query, cb) -> {
            active == null ? null : cb.equal(root.get("active"), active);
        };
    }

    public static Specification<User> ageBetween(Integer min, Integer max) {
        return (root, query, cb) -> {
            if (min == null && max == null) return null;
            if (min == null) return cb.lessThanOrEqualTo(root.get("age"), max);
            if (max == null) return cb.greaterThanOrEqualTo(root.get("age"), min);
            return cb.between(root.get("age"), min, max);
        };
    }

    public static Specification<User> createdAfter(LocalDate date) {
```

```
return (root query ch) -> {  
    date == null ? null : ch.greaterThanOrEqualTo(  
        root.get("createdAt"), date.atStartOfDay())  
    }  
  
    // Join-based specification: filter by department name (related entity)  
    public static Specification<User> inDepartment(String deptName) {  
        return (root query ch) -> {  
            if (deptName == null) return null;  
            Join<User, Department> dept = root.join("department", JoinType.LEFT);  
            return ch.equal(dept.get("name"), deptName).  
        };  
    }  
  
}  
  
// ■■■ Repository — must extend JpaSpecificationExecutor  
public interface UserRepository extends  
    JpaRepository<User, Long>  
    ,JpaSpecificationExecutor<User> {}  
  
// ■■■ Service — compose specifications dynamically  
public Page<User> searchUsers(UserSearchRequest req, Pageable pageable) {  
    Specification<User> spec = Specification.where(hasName(req.getName()))  
        .and(hasRole(req.getRole()))  
        .and(isActive(req.getActive()))  
        .and(ageBetween(req.getMinAge(), req.getMaxAge()))  
        .and(createdAfter(req.getCreatedAfter()))  
        .and(inDepartment(req.getDepartment()));  
    return userRepository.findAll(spec, pageable).  
}
```

■ **WHY THIS MATTERS:** Each Specification is independently unit-testable. You can test `hasRole()` with a mock `CriteriaBuilder` without hitting a database. Specifications are also reusable across different service methods — `'isActive()'` can be combined with completely different specifications elsewhere in the codebase.

CHAPTER 8

Transactions in Spring Data JPA

8.1 What Is a Transaction and Why Is It Non-Negotiable?

A database transaction is a sequence of one or more SQL operations that are treated as a single, indivisible unit of work. The fundamental rule is: either ALL operations in the transaction succeed and are permanently committed to the database, or NONE of them are — the database is rolled back to the exact state it was in before the transaction started.

Consider a bank transfer: you need to debit \$500 from Account A and credit \$500 to Account B. This requires two UPDATE statements. Without a transaction, if your application crashes between the first UPDATE (debit) and the second UPDATE (credit), you have successfully removed \$500 from Account A but never added it to Account B. The money is gone. With a transaction, the partial state is never committed — a crash causes an automatic rollback and Account A keeps its \$500. This is why transactions are fundamental to any system that modifies data.

- **Atomicity:** The transaction is atomic — all operations succeed together, or none do. There is no partial success.
- **Consistency:** The transaction brings the database from one valid state to another. Constraints (NOT NULL, foreign keys, unique indexes) are checked and must be satisfied at commit time.
- **Isolation:** Concurrent transactions execute as if they were isolated from each other. One transaction should not see the uncommitted changes of another transaction (depending on isolation level).
- **Durability:** Once a transaction is committed, it remains committed even if the database server crashes immediately afterward. The data is written to durable storage (disk).

8.2 @Transactional — How the Spring Proxy Mechanism Works

When you annotate a method or class with `@Transactional`, Spring does not modify your class directly. Instead, Spring creates a proxy — either a JDK dynamic proxy (if your class implements an interface) or a CGLIB subclass (if it does not). This proxy wraps your class. When another Spring bean calls a `@Transactional` method, it actually calls the proxy's method, not your method directly.

The proxy's method does the following: it asks the `TransactionInterceptor` to check if there is already an active transaction in the current thread (stored in a `ThreadLocal` variable). Based on the propagation setting, it either joins the existing transaction or creates a new one by calling `PlatformTransactionManager.getTransaction()`. It then calls your actual method. If your method completes without throwing an exception, the proxy calls `transactionManager.commit()`. If your method throws a `RuntimeException` (or `Error`), the proxy calls `transactionManager.rollback()`. If it throws a checked exception, by default the proxy still commits (this surprises many developers).

`@Transactional` internals visualised

// What your code looks like:

```

@Service
public class OrderService {
    @Transactional
    public Order placeOrder(OrderRequest req) {
        Order order = orderRepo.save(new Order(req));
        inventoryService deductStock(req.getItems()); // may throw
        paymentService charge(req.getPayment()); // may throw
        return order;
    }
}

// What Spring's proxy ACTUALLY executes (conceptually):
public Order placeOrder(OrderRequest req) { // This is the PROXY method
    TransactionStatus tx = txManager.getTransaction(new DefaultTransactionDef());
    try {
        Order result = super.placeOrder(req); // calls your real method
        txManager.commit(tx);
        return result;
    } catch (RuntimeException e) {
        txManager.rollback(tx); // only rolls back for RuntimeException!
        throw e;
    }
    // CHECKED exceptions cause commit by default — see rollbackFor below
}

```

■ **WATCH OUT:** `@Transactional` ONLY rolls back for unchecked exceptions (`RuntimeException` and its subclasses) and `Error` by default. If your method throws a checked exception (like `IOException` or `SQLException`), Spring will COMMIT the transaction. Always use `@Transactional(rollbackFor = Exception.class)` if your method can throw checked exceptions that should cause a rollback.

8.3 Propagation Types — The Full Guide

Propagation controls what happens when a `@Transactional` method is called while another transaction may already be active. This is critical when you have service methods calling other service methods. There are 7 propagation types, and knowing when to use each is an advanced interview topic.

REQUIRED (the default — used 95% of the time)

If there is already an active transaction, the method joins it and runs within the same transaction. If there is no active transaction, a new one is created. This is the safe, sensible default. It ensures that if Service A calls Service B, and both are `REQUIRED`, they run in the same transaction — if B throws an exception, the entire transaction (including A's work) is rolled back together.

REQUIRES_NEW (used for audit logging and independent writes)

`REQUIRES_NEW` always suspends any existing transaction and creates a brand-new, completely independent transaction. This new transaction commits or rolls back independently of the outer transaction. This is the right choice for audit logging: you want the audit record (who did what, when) to be written to the database even if the main business transaction rolls back. If you used `REQUIRED` for auditing, a failed business operation would also roll back the audit record, leaving you with no trace of what happened.

MANDATORY (enforces 'must be called within a transaction')

MANDATORY throws a `TransactionRequiredException` if there is no active transaction. This is a design-by-contract annotation — you are saying: 'this method is a helper that should ONLY ever be called from within an existing transaction. If someone calls it without a transaction, that is a programming error, and I want an exception.' This prevents accidental non-transactional calls to critical database operations.

Propagation examples

```
// REQUIRED: default — join existing or create new
@Transactional(propagation = Propagation.REQUIRED)
public void createOrder(Order order) { }

// REQUIRES_NEW: audit always committed even if outer TX rolls back
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logAuditEvent(String event) {
    auditRepo.save(new AuditLog(event, LocalDateTime.now()));
} // this commit is independent — outer rollback does NOT affect it

// MANDATORY: this helper method MUST be called within an active TX
@Transactional(propagation = Propagation.MANDATORY)
public void validateAndLockInventory(List<OrderItem> items) {
    // If called without active TX throws TransactionRequiredException
    items.forEach(item -> inventoryRepo.lockForUpdate(item.getProductId()));
}

// NOT_SUPPORTED: suspend any TX — run completely non-transactionally
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public List<Product> getBestSellersReport() {
    // Long-running read: don't hold a TX open during this query
    return reportingService.computeBestSellers();
}
```

8.4 Read-Only Transactions — A Must-Know Optimisation

Marking a `@Transactional` method as `readOnly = true` is not just documentation — it has real performance implications. When Hibernate knows a transaction is read-only, it skips the dirty-checking phase entirely. Normally, at flush time, Hibernate compares every managed entity's current state to its snapshot taken at load time, looking for changes. In a query-heavy service method that loads hundreds of entities, this snapshot comparison is significant CPU work. With `readOnly = true`, Hibernate skips it because you have declared that no modifications will happen.

Additionally, some database connection routing setups (read replica configurations) use the `readOnly` hint to direct queries to a read replica database instead of the primary database. This horizontally scales your read capacity without any code changes beyond adding `readOnly = true`.

Read-only transaction best practice pattern

```
// Best practice: annotate the whole service class as readOnly=true
// then override individual write methods with readOnly=false (the default)
@Service
@Transactional(readOnly = true) // All methods read-only by default
public class UserService {
    public List<UserDto> getAllUsers() {
        // Inherits readOnly=true — dirty checking skipped
        return userRepo.findAll().stream().map(UserDto::from).toList();
    }

    public Page<UserDto> searchUsers(UserFilter f, Pageable p) {
        // Inherits readOnly=true
        return userRepo.findAll(UserSpecs.build(f), p).map(UserDto::from);
    }
}
```

```

@Transactional // Overrides class-level readOnly=true with readOnly=false
public UserDto createUser(CreateUserRequest req) {
    User user = new User(req.getName(), req.getEmail());
    return UserDto.from(userRepo.save(user));
}

@Transactional // Write method – overrides readOnly
public UserDto updateUser(Long id, UpdateUserRequest req) {
    User user = userRepo.findById(id).orElseThrow();
    user.setName(req.getName()); // dirty checking WILL run here
    return UserDto.from(user); // no explicit save() needed!
}
}

```

8.5 The Self-Invocation Trap — Why @Transactional Sometimes Does Nothing

This is one of the most common and confusing bugs in Spring applications. Because `@Transactional` works through a proxy, it only intercepts calls that come from outside the bean. If a method within the same class calls another `@Transactional` method on 'this', the call bypasses the proxy entirely — no transaction is started, no transaction is joined. The annotation is completely ignored.

Self-invocation anti-pattern and fixes

[illegible]

■ **WATCH OUT:** The self-invocation problem is silent — Spring will not warn you that `@Transactional` is being ignored. You will discover it only when you see data inconsistency in production or when a test fails because a rollback did not happen. Always extract methods that need their own transaction into a separate Spring bean.

8.6 Optimistic vs Pessimistic Locking

When multiple users can update the same record concurrently, you need a locking strategy to prevent lost updates. Without locking, User A and User B both read the same record, User A saves changes, then User B saves their changes — overwriting User A's changes without knowing they existed. This is called the lost update problem.

Optimistic Locking — Assume No Conflict, Detect If One Occurs

Optimistic locking assumes conflicts are rare. It does not lock the database row at read time. Instead, it adds a version column to the entity. Every time a row is updated, the version number increments. When Hibernate generates the UPDATE statement, it includes WHERE version = :originalVersion in the WHERE clause. If another transaction already updated the row (incrementing the version), this WHERE clause matches zero rows, and Hibernate throws an OptimisticLockException. Your application code can then decide whether to retry the operation or show an error to the user.

Pessimistic Locking — Lock the Row Before You Read It

Pessimistic locking assumes conflicts are likely and locks the database row immediately upon reading it with `SELECT ... FOR UPDATE`. No other transaction can modify (or even read, with `PESSIMISTIC_WRITE`) that row until your transaction releases the lock. This prevents lost updates but has costs: it reduces throughput (other transactions must wait), it can cause deadlocks (Transaction A holds Lock 1 and waits for Lock 2; Transaction B holds Lock 2 and waits for Lock 1), and it holds database connections and resources for the duration of the lock. Use pessimistic locking only when the cost of a conflict is very high (e.g., booking a seat, reducing inventory) and conflicts are genuinely common.

Optimistic and pessimistic locking examples

[illegible]

```
public BookingConfirmation bookSeat(Long seatId, Long userId) {  
    // Locks the row – other transactions BLOCK here until we commit  
    Seat seat = seatRepo.findByIdForUpdate(seatId).orElseThrow();  
    if (seat.isBooked()) throw new SeatAlreadyBookedException();  
    seat.setBooked(true);  
    seat.setUserId(userId);  
    return new BookingConfirmation(seat);  
} // lock released on commit
```

CHAPTER

Quick-Reference Cheatsheet & Top Interview Q&A;

Annotations at a Glance

Annotation	Purpose / Explanation
@Entity	Marks the class as a JPA-managed entity — Hibernate will map it to a DB table
@Table(name=...)	Override the default table name; add DB-level indexes and unique constraints
@Id	Marks the primary key field — every entity must have exactly one
@GeneratedValue	Tells Hibernate how to generate PK values — IDENTITY, SEQUENCE, TABLE, UUID
@Column(...)	Map field to specific column; set nullable, length, unique, updatable constraints
@Embeddable + @Embedded	Value object stored in the owner's table — no separate table or FK needed
@Enumerated(String)	Always use String — stores enum name; ORDINAL corrupts data on enum reorder
@OneToMany(mappedBy)	Inverse side of a One-to-Many; mappedBy points to the FK owner field
@ManyToOne + @JoinColumn	Owning side of a Many-to-One; this entity's table has the FK column
@ManyToMany + @JoinTable	Many-to-many with an explicit junction table
@Version	Optimistic locking — Hibernate increments this on every UPDATE
@Transactional	Declares a transaction boundary; Spring proxy manages begin/commit/rollback
@Transactional(readOnly=true)	Skips dirty checking; may route to read replica — use for all read methods
@Transactional(rollbackFor=Exception.class)	Rollback checked exceptions too (default only rolls back RuntimeException)
@Query(nativeQuery=true)	Execute raw SQL — uses table/column names, not entity/field names
@Modifying	Required on any @Query that is an UPDATE or DELETE statement
@Lock(PessimisticWrite)	Adds SELECT ... FOR UPDATE to lock the row until transaction commits
@EntityGraph	Override lazy fetch for a specific method — avoids N+1 without JOIN FETCH
@PageableDefault	Set default page size/sort for a controller's Pageable parameter

Top 12 Interview Questions — Answered in Full

Q1. What is the difference between JPA, Hibernate, and Spring Data JPA?

JPA (Jakarta Persistence API) is a specification — a set of interfaces and annotations (like @Entity, @Id) that define HOW Java objects should be mapped to database tables, but it contains no executable code. Hibernate is a JPA provider — it is the actual implementation that generates SQL,

manages the persistence context, handles caching and lazy loading, and talks to the database. Spring Data JPA is a higher-level abstraction built on top of both — it automatically generates repository implementations so you never have to write DAO classes, provides derived query methods from method names, and integrates JPA with Spring's dependency injection and transaction management.

Q2. What is the N+1 query problem and how do you fix it?

The N+1 problem occurs when loading N entities triggers N additional queries to load a lazy association on each entity — resulting in N+1 total queries instead of 1 or 2. For example, loading 100 Orders and then accessing `getItems()` on each one fires $1 + 100 = 101$ queries. The fix is to use `JOIN FETCH` in a `@Query` annotation to load the association in a single JOIN query, or use `@EntityGraph` to declaratively override lazy loading for a specific method. DTO projections that exclude the association entirely are also a valid solution if you do not need the children.

Q3. How does the @Transactional proxy work — and what is self-invocation?

Spring creates a proxy (JDK dynamic proxy or CGLIB subclass) around your bean. When external code calls a `@Transactional` method, the proxy intercepts it, opens a transaction, calls your real method, and commits or rolls back based on exceptions. Self-invocation is when a method within the same class calls another `@Transactional` method using `'this.method()'` — this call goes directly to your object, bypassing the proxy entirely. No transaction is started. The fix is to extract the inner method to a separate Spring bean so the call goes through the proxy.

Q4. Why should you always use @Enumerated(EnumType.STRING)?

`EnumType.ORDINAL` stores the enum's position in its declaration as an integer (0, 1, 2...). If you later add a new enum value in the middle of the declaration, all subsequent ordinal values shift. Existing database records now map to the wrong enum constant — silently corrupting your data with no error or warning. `EnumType.STRING` stores the enum name as a VARCHAR ('ADMIN', 'USER'), so adding enum values anywhere does not affect existing data.

Q5. What is LAZY vs EAGER fetching and why is LAZY preferred?

LAZY loading means an association is not loaded from the database until you first access it — Hibernate creates a proxy placeholder. EAGER loading means the association is loaded immediately via a JOIN whenever you load the parent entity, even if you never use the association. LAZY is preferred because it avoids loading data you do not need, reducing database I/O and memory usage. You should start with all associations LAZY and add `JOIN FETCH` or `@EntityGraph` only where you actually need the association in a specific query.

Q6. What is the difference between Page and Slice?

Page<T> executes two queries: one to fetch the current page of results and one COUNT(*) query to get the total number of matching records. This gives you totalElements and totalPages, needed for a traditional pagination UI. Slice<T> executes only one query — it fetches pageSize+1 rows and uses the extra row to determine whether hasNext() is true. Slice is faster (no COUNT) and is the right choice for infinite-scroll or 'Load More' UIs where the total count is not displayed.

Q7. What is CascadeType.ALL and when is it dangerous?

CascadeType.ALL means cascade all operations (PERSIST, MERGE, REMOVE, REFRESH, DETACH) from the parent entity to its children. It is appropriate on @OneToMany when the children have no meaning outside the parent (e.g., OrderItems belong only to an Order). It is DANGEROUS on @ManyToMany — cascading REMOVE through a many-to-many means deleting one entity cascades to the junction table and potentially to the other side's entities, which can delete records that other entities also reference.

Q8. What do propagation types do? Give examples of REQUIRED and REQUIRES_NEW.

Propagation controls what happens when a @Transactional method is called while another transaction may already be active. REQUIRED (the default) joins an existing transaction if one is active, or creates a new one — meaning if Service A calls Service B and both are REQUIRED, they share one transaction and one rollback affects both. REQUIRES_NEW always suspends the outer transaction and creates a new independent transaction — useful for audit logging, where you want the audit record committed even if the outer business transaction rolls back.

Q9. What does readOnly=true on @Transactional do?

It tells Hibernate to skip dirty checking — normally Hibernate compares every managed entity's current state to its loaded snapshot before flushing, looking for changes. With readOnly=true, this comparison is skipped because you have declared no modifications will occur, saving CPU on large result sets. It also signals to connection routing layers that the query can be served by a read-only database replica, horizontally scaling read capacity.

Q10. What is optimistic locking and when does it throw an exception?

Optimistic locking uses a @Version column (integer or timestamp) on the entity. Hibernate adds AND version = :originalVersion to every UPDATE statement. If another transaction already committed an update to the same row (incrementing the version), the WHERE clause matches zero rows, and Hibernate throws OptimisticLockingFailureException. Optimistic locking is preferred for high-read, low-conflict scenarios because it does not hold any database locks during the read phase — the conflict check only happens at commit time.

Q11. What is the LazyInitializationException and how do you prevent it?

`LazyInitializationException` is thrown by Hibernate when you try to access a lazy-loaded association after the persistence context (`EntityManager` / Hibernate session) has been closed. Typically this happens when a controller or Jackson serializer tries to access a lazy collection on an entity that was loaded in a service method that has already returned (and therefore the transaction and session are closed). Prevention: use `JOIN FETCH` or `@EntityGraph` to load needed associations inside the transaction, use DTO projections that only include the fields you need, or (as a last resort) use `@Transactional` on the controller — though that is generally bad practice.

Q12. What is the difference between `save()` and `saveAndFlush()` in `JpaRepository`?

`save()` persists or merges the entity and marks it for synchronisation with the database, but the actual SQL INSERT or UPDATE may not execute immediately — it is deferred until Hibernate flushes the persistence context (usually at the end of the transaction). `saveAndFlush()` immediately writes the change to the database within the current transaction, making it visible to native queries or JDBC operations within the same transaction. Use `saveAndFlush()` when you need to immediately execute the SQL and see the result in the same transaction, for example before calling a native stored procedure.